

C++ MATH MASTERY

Standard & Advanced Mathematical Libraries

CodeHive Academy

Module 6: Computational Logic

1. Comparison: max() and min()

In **CodeHive's** introductory logic, we often need to choose between two data points. C++ provides two simple built-in functions to find the extremes without needing long if-else statements.

- `max(a, b)`: Returns the larger of the two values.
- `min(a, b)`: Returns the smaller of the two values.

```
#include <iostream> using namespace std; int main() { int a = 5, b = 10;
cout << "Maximum: " << max(a, b) << endl; cout << "Minimum: " << min(a, b)
<< endl; return 0; }
```

```
Maximum: 10
Minimum: 5
```

2. The <cmath> Powerhouse

For operations beyond simple comparisons, CodeHive developers utilize the <cmath> library. This library links your code to high-performance mathematical algorithms.

Square Roots and Power

```
cout << sqrt(64); // Result: 8 cout << pow(2, 5); // 2 to the power of 5 = 32
```

CodeHive Engineering Tip: `sqrt()` and `pow()` always work with **double** (decimal) values by default. If you store the result in an `int`, the decimals will be cut off!

3. Precision and Rounding

When working with decimals (floats/doubles), you often need to transform them into whole numbers. C++ provides three distinct ways to do this.

- **round(x)**: Rounds to the nearest whole integer (.5 and up goes up).
- **ceil(x)**: Always rounds **UP** (towards the ceiling).
- **floor(x)**: Always rounds **DOWN** (towards the floor).

```
cout << round(2.6); // Result: 3 cout << ceil(2.1); // Result: 3 cout <<
floor(2.9); // Result: 2
```

4. Absolute Values and Logarithms

Calculations in data science and game physics often require handling negative numbers or exponential growth.

The Absolute Value: `abs(x)`

Removes the negative sign. `abs(-7)` becomes 7. This is vital for calculating distances between two points.

The Natural Log: `log(x)`

In C++, `log()` performs the Natural Logarithm (ln). If you need Log base 10, you must use `log10()`.

```
cout << abs(-7); // 7 cout << log(2.71); // approx 1.0
```

5. Smart Math Combinations

Professionals at CodeHive don't just use these functions alone; we nest them to create powerful one-line solutions.

```
double sideA = sqrt(81); // 9 double sideB = pow(2, 4); // 16 // Find the
largest result between two complex formulas cout << "Winner: " <<
max(sideA, sideB);
```

Checklist:

1. Always `#include <cmath>`
2. Expect decimal results from `cmath` functions.
3. Use `abs()` to safeguard against negative distance errors.

6. Practice & Summary

Congratulations! You can now perform advanced calculations. Use these challenges to cement your knowledge.

🔧 CodeHive Lab Challenges

1. **Hypotenuse Calculator:** Ask for sides A and B. Calculate `sqrt(pow(A, 2) + pow(B, 2))`.
2. **Rounder:** Take a decimal input and show the `ceil`, `floor`, and `round` values simultaneously.
3. **Power Play:** Ask for a base and an exponent, then display the result.

